CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT

ON

To-Do List Web Application

Program Name: BCA

Subject Name/Code: Web Designing (24CAH-153)

Submitted by:

Name: Mohita

UID: 24BCA10307

Section: 24BCA6

Group: A

Submitted to:

Name: Mr. Jagvinder Singh

Designation: Assistant Professor

Contents

1. Introduction 3

2. Objectives 3

3. Tools & Technologies Used 3

4. System Requirements 3

Hardware Requirements: 3

Software Requirements: 4

5. System Design & Implementation 4

6. Testing & Validation 4

7. Code Implementation 5

8. Results & Discussion 9

9. Future Enhancements 9

10. Conclusion 9

1. Introduction

The To-Do List Website is a simple yet functional web-based application that allows users to manage daily tasks efficiently. Users can add, mark complete, and delete tasks. The project demonstrates the effective use of HTML, CSS, and JavaScript to create an interactive and visually appealing user experience. It is designed for beginners to understand basic web development concepts and for users to practice organizing tasks digitally.

---

2. Objectives

- To design a user-friendly web interface for task management.
- To implement functionality for adding, completing, and deleting tasks.
- To use localStorage to persist data across browser sessions.
- To apply modern styling techniques for professional UI/UX.
- To demonstrate client-side scripting using JavaScript.

---

3. Tools & Technologies Used

- **HTML5** – for structuring the content.
- **CSS3** – for styling and layout design with modern visual appeal (e.g., gradients, glassmorphism).
- **JavaScript** – for implementing interactive functionality.
- **LocalStorage API** – for storing tasks locally on the user's browser.
- **Text Editor** – Visual Studio Code or any other preferred IDE.
- **Browser** – Google Chrome, Mozilla Firefox, etc., for testing and running the app.

---

4. System Requirements

Hardware Requirements:

- Processor: Intel Core i3 or higher

- RAM: 2 GB minimum
- Storage: 500 MB free disk space
- Display: 1024x768 resolution or higher

Software Requirements:

- Web Browser (Chrome, Firefox, Edge, etc.)
- Code Editor (e.g., VS Code, Sublime Text)
- Operating System: Windows / Linux / macOS

---

5. System Design & Implementation

The project consists of a single HTML file that combines structure, styling, and functionality. The interface includes:

- An input field to enter tasks.
- An "Add" button to append the task to the list.
- A dynamic task list where each item can be marked complete or deleted.
- Tasks are stored in the browser using local Storage so that data is retained even after refreshing.

Key Features:

- Responsive layout
- Hover and click transitions
- Visual feedback for completed tasks
- Scrollable task list with custom scrollbar

---

6. Testing & Validation

The application was manually tested in the following ways:

- Adding tasks with valid and empty input
- Marking tasks as completed
- Deleting tasks
- Refreshing the page to ensure task persistence
- Testing across different browsers to verify compatibility
- Validating code using W3C Validator for HTML/CSS standards

All core features worked as expected without any major bugs.

7. Code Implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>To-Do List</title>
  <style>
    /* --- Reset + Base Styles --- */
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #74ebd5, #acb6e5);
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
    }

    /* --- Container --- */
    .todo-container {
      background: rgba(255, 255, 255, 0.15);
      backdrop-filter: blur(12px);
      -webkit-backdrop-filter: blur(12px);
      border-radius: 20px;
      box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.2);
      padding: 2rem;
      width: 90%;
      max-width: 450px;
      color: #fff;
      transition: all 0.3s ease-in-out;
    }

    h1 {
      text-align: center;
      margin-bottom: 1.5rem;
      font-size: 2rem;
      font-weight: 600;
    }

    /* --- Input Group --- */
    .input-group {
```

```
  display: flex;
  gap: 0.5rem;
  margin-bottom: 1.2rem;
}

#taskInput {
  flex: 1;
  padding: 0.6rem 1rem;
  border-radius: 12px;
  border: none;
  font-size: 1rem;
}

#addBtn {
  padding: 0.6rem 1rem;
  background: #4e54c8;
  color: white;
  border: none;
  border-radius: 12px;
  font-weight: bold;
  cursor: pointer;
  transition: background 0.3s;
}

#addBtn:hover {
  background: #565ce2;
}

/* --- Task List --- */
ul {
  list-style: none;
  padding: 0;
  max-height: 250px;
  overflow-y: auto;
}

li {
  background: rgba(255, 255, 255, 0.2);
  padding: 0.7rem 1rem;
  margin-bottom: 0.7rem;
  border-radius: 10px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: background 0.3s;
  cursor: pointer;
}

li.completed {
  text-decoration: line-through;
  color: #ddd;
```

```
      background: rgba(200, 200, 200, 0.2);
    }

    .deleteBtn {
      background: #ff4d4d;
      border: none;
      color: white;
      padding: 0.3rem 0.6rem;
      border-radius: 6px;
      font-size: 0.9rem;
      cursor: pointer;
      transition: background 0.2s;
    }

    .deleteBtn:hover {
      background: #e60000;
    }

    /* --- Scrollbar Styling --- */
    ul::-webkit-scrollbar {
      width: 6px;
    }

    ul::-webkit-scrollbar-thumb {
      background: #ffffff55;
      border-radius: 10px;
    }
  </style>
</head>
<body>
  <div class="todo-container">
    <h1>📝 To-Do List</h1>
    <div class="input-group">
      <input type="text" id="taskInput" placeholder="Enter a
task...">
      <button id="addBtn">Add</button>
    </div>
    <ul id="taskList"></ul>
  </div>

  <script>
    const taskInput = document.getElementById("taskInput");
    const addBtn = document.getElementById("addBtn");
    const taskList = document.getElementById("taskList");

    // Load tasks from localStorage
    window.onload = function () {
      const savedTasks =
JSON.parse(localStorage.getItem("tasks")) || [];
      savedTasks.forEach(task => {
        createTaskElement(task.text, task.completed);
```

```
    });
  };

  addBtn.addEventListener("click", () => {
    const taskText = taskInput.value.trim();
    if (taskText !== "") {
      createTaskElement(taskText);
      saveTask(taskText, false);
      taskInput.value = "";
    }
  });

  function createTaskElement(text, completed = false) {
    const li = document.createElement("li");
    li.textContent = text;

    if (completed) {
      li.classList.add("completed");
    }

    li.addEventListener("click", () => {
      li.classList.toggle("completed");
      updateTasks();
    });

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.classList.add("deleteBtn");
    deleteBtn.addEventListener("click", (e) => {
      e.stopPropagation(); // prevent toggle
      li.remove();
      updateTasks();
    });

    li.appendChild(deleteBtn);
    taskList.appendChild(li);
  }

  function saveTask(text, completed) {
    const tasks = JSON.parse(localStorage.getItem("tasks"))
|| [];
    tasks.push({ text, completed });
    localStorage.setItem("tasks", JSON.stringify(tasks));
  }

  function updateTasks() {
    const tasks = [];
    document.querySelectorAll("#taskList li").forEach(li => {
      tasks.push({
        text: li.firstChild.textContent,
        completed: li.classList.contains("completed")
```

```
        });
      });
      localStorage.setItem("tasks", JSON.stringify(tasks));
    }
  </script>
</body>
</html>
```

---

8. Results & Discussion

The To-Do List website successfully met its objectives. Users are able to manage tasks efficiently with a simple interface. The use of localStorage enhances user experience by maintaining data persistency. The modern UI design adds visual appeal and helps in better engagement.

Challenges faced:

- Handling edge cases like duplicate entries
- Styling for consistent appearance across browsers

---

9. Future Enhancements

- Add categories or tags for task grouping
- Include due dates and reminders using the Date API
- Enable drag-and-drop functionality for reordering tasks
- Add user authentication to support multiple users
- Sync tasks to cloud storage or a backend database

---

10. Conclusion

This mini project demonstrates how core web technologies (HTML, CSS, and JavaScript) can be used together to build a useful and elegant application. The project strengthened understanding of DOM manipulation, event handling, and data storage in the browser. It serves as a strong foundation for further development of full-fledged task management systems.